



课堂回顾



Roofline模型

横轴: flops/byte

纵轴: flops/second

- **Roofline:**
An insightful Visual Performance model for multicore Architectures

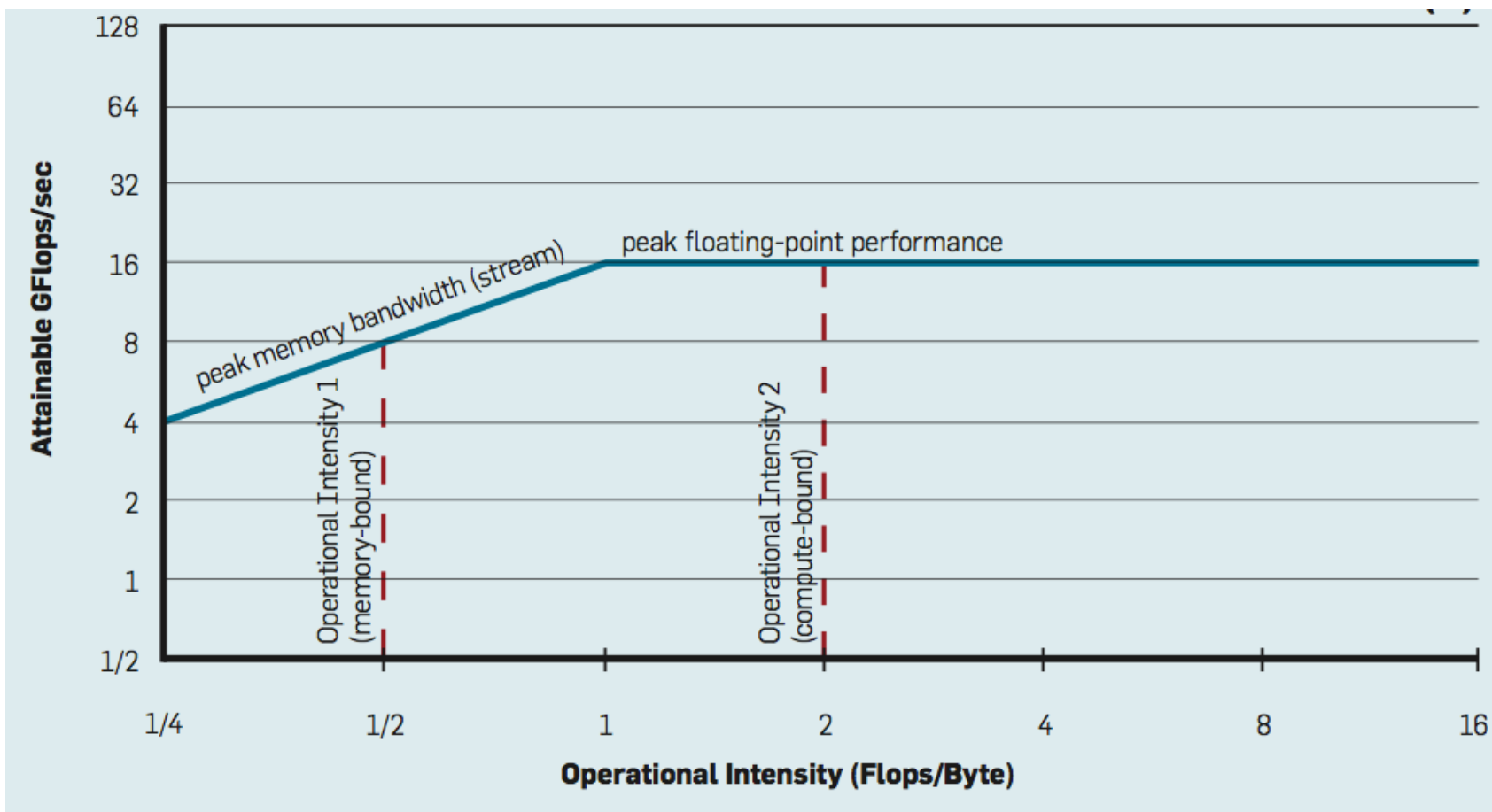


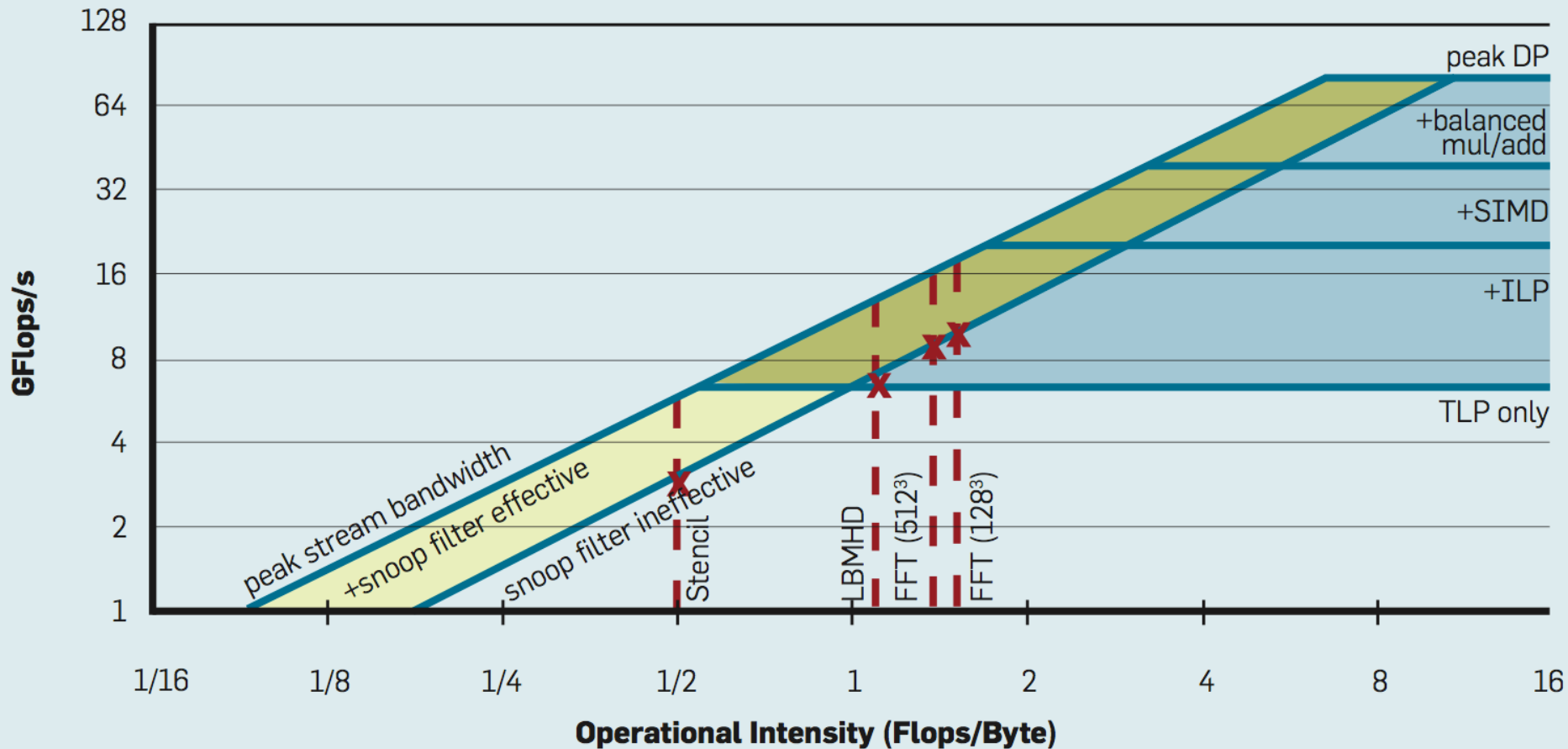
Table 2: Characteristics of four floating-point kernels.

Name	Operational Intensity	Description
SpMV ²⁹	0.17 to 0.25	Sparse Matrix-Vector multiply: $y = A \cdot x$ where A is a sparse matrix and x, y are dense vectors; multiplies and adds equal.
LBMHD ²⁸	0.70 to 1.07	Lattice-Boltzmann Magnetohydro-dynamics is a structured grid code with a series of time steps.
Stencil ¹²	0.33 to 0.50	A multigrid kernel that updates seven nearby points in a 3D stencil for a 256^3 problem.
3D FFT	1.09 to 1.64	3D Fast Fourier Transform (2 sizes: 128^3 and 512^3).

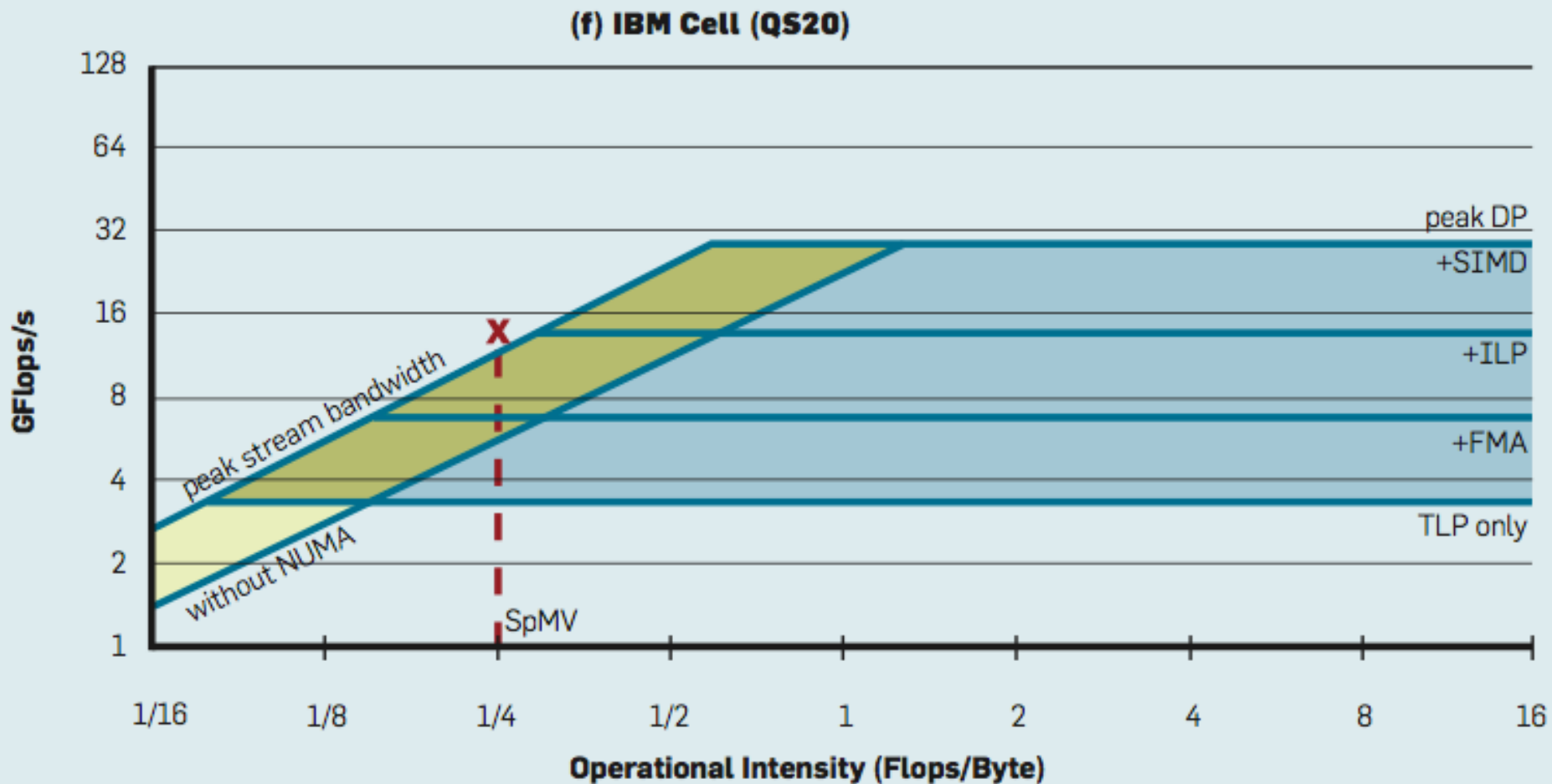


ROOFLINE MODEL

(a) Intel Xeon (Clovertown)



ROOFLINE MODEL





CUDA介绍

谢旻晖



中國人民大學
RENMIN UNIVERSITY OF CHINA



CUDA教材

- ParallelLab:/usr/local/cuda/doc/pdf/
- 入门: CUDA_C_Programming_Guide.pdf
- 优化: CUDA_C_Best_Practices_Guide.pdf
- 本节课专注于CUDA语言, 不会涉及GPU内体系结构具体细节, GPU体系结构下节课讲

目录

**Heterogeneous
Computing**

Blocks

Threads

Indexing

Shared memory

`_syncthreads()`

**Asynchronous
operation**

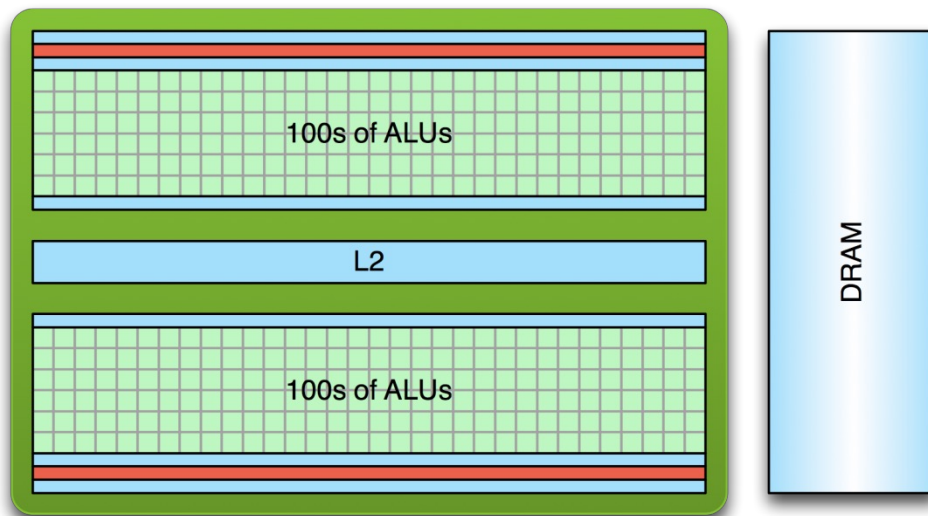
Handling errors

Managing devices



GPU计算

- GPU: Graphics Processing Unit
- 较高的计算密度（大量的ALU），高内存带宽
- Throughput processor: 上千个线程同时运算
隐藏延迟（和快速cache相对应）





CUDA

- CUDA
 - 用GPU做常规计算任务
 - 较高性能
- CUDA C/C++
 - 基于规范的C / C++语法
 - 较小的扩展来支持异构计算
 - API直接管理设备、内存等



CUDA C / C++

- 本节课学习什么内容？
 - 从Hello Word程序开始学习CUDA
 - 写CUDA C / C++ kernel
 - 使用GPU内存资源
 - 通信和同步操作



要求

- 有一些C / C++经验
- 不需要有GPU经验
- 不需要有图形图像经验
- 服务器设定路径：
 - vim ~/.bashrc
 - 最后加入一行：
 - export PATH=\$PATH:/usr/local/cuda/bin

```
feng@ParallelLab:~/NVIDIA_CUDA-9.0_Samples/bin/x86_64/linux/release$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2017 NVIDIA Corporation
Built on Fri_Sep__1_21:08:03_CDT_2017
Cuda compilation tools, release 9.0, V9.0.176
```

CONCEPTS

Heterogeneous
Computing

Blocks

Threads

Indexing

Shared memory

`_syncthreads()`

Asynchronous
operation

Handling errors

Managing devices



Heterogeneous Computing

Blocks

Threads

Indexing

Shared memory

`_syncthreads()`

Asynchronous
operation

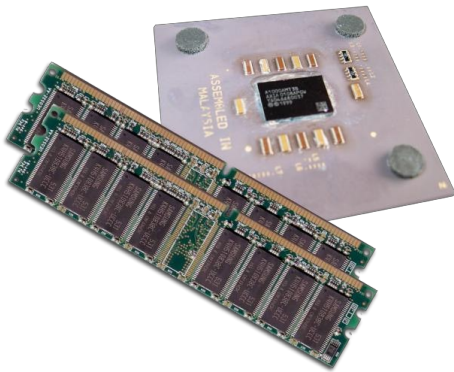
Handling errors

Managing devices

- HELLO WORD!

异构计算

- 术语:
- HOST CPU和CPU的memory (host memory)
- DEVICE GPU和GPU的memory(device memory)



Host



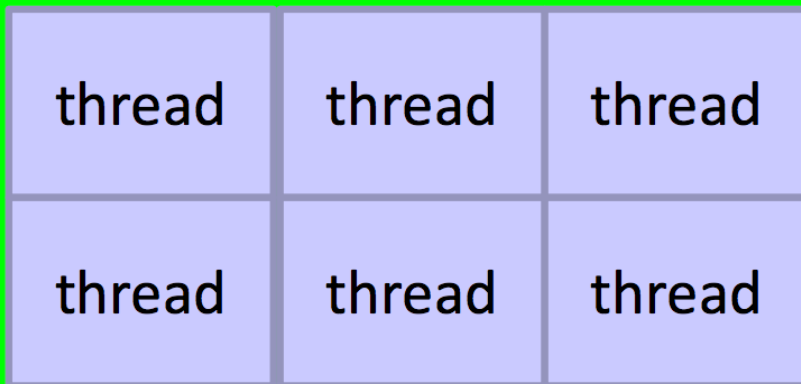
Device



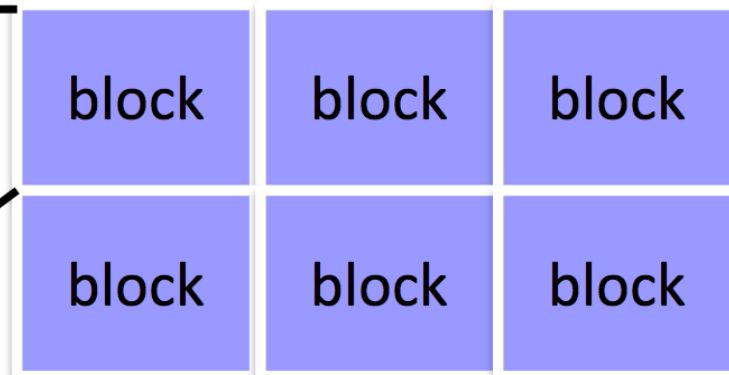
CUDA术语

- Host: CPU
- Device: GPU
- Kernel: GPU上的function
- Thread: 基本执行单元
- Block: 一组线程
- Grid: 一组block

Block



Grid



异构计算

```
#include <iostream>
#include <algorithm>

using namespace std;

#define N 1024
#define RADIUS 3
#define BLOCK_SIZE 16

__global__ void stencil_1d(int *in, int *out) {
    __shared__ int temp[BLOCK_SIZE * 2 * RADIUS]; int
    gindex = threadIdx.x + blockIdx.x * blockDim.x; int
    lindex = threadIdx.x + RADIUS;

    // Read input elements into shared memory
    temp[lindex] = in[gindex];
    if (threadIdx.x < RADIUS) {
        temp[lindex - RADIUS] = in[gindex - RADIUS]; temp[lindex
        + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }

    // Synchronize (ensure all the data is available)
    __syncthreads();

    // Apply the stencil
    int result = 0;
    for (int offset = -RADIUS; offset <= RADIUS; offset++)
        result += temp[lindex + offset];

    // Store the result
    out[gindex] = result;
}

void fill_ints(int *x, int n) {
    fill_n(x, n, 1);
}

int main(void) {
    int *in, *out; // host copies of a, b, c
    int *d_in, *d_out; // device copies of a, b, c
    int size = (N + 2 * RADIUS) * sizeof(int);

    // Alloc space for host copies and setup values
    in = (int *) malloc(size); fill_ints(in, N + 2 * RADIUS); out
    = (int *) malloc(size); fill_ints(out, N + 2 * RADIUS);

    // Alloc space for device copies
    cudaMalloc((void **) &d_in, size);
    cudaMalloc((void **) &d_out, size);

    // Copy to device
    cudaMemcpy(d_in, in, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_out, out, size, cudaMemcpyHostToDevice);

    // Launch stencil_1d() kernel on GPU
    stencil_1d<<<N/BLOCK_SIZE, BLOCK_SIZE>>>>(d_in + RADIUS, d_out + RADIUS);

    // Copy result back to host
    cudaMemcpy(out, d_out, size, cudaMemcpyDeviceToHost);

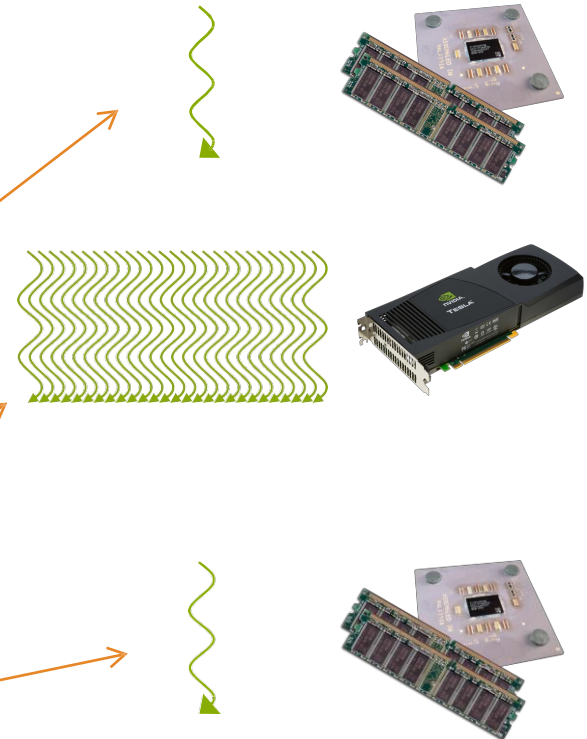
    // Cleanup
    free(in); free(out);
    cudaFree(d_in); cudaFree(d_out);
    return 0;
}
```

parallel
fn

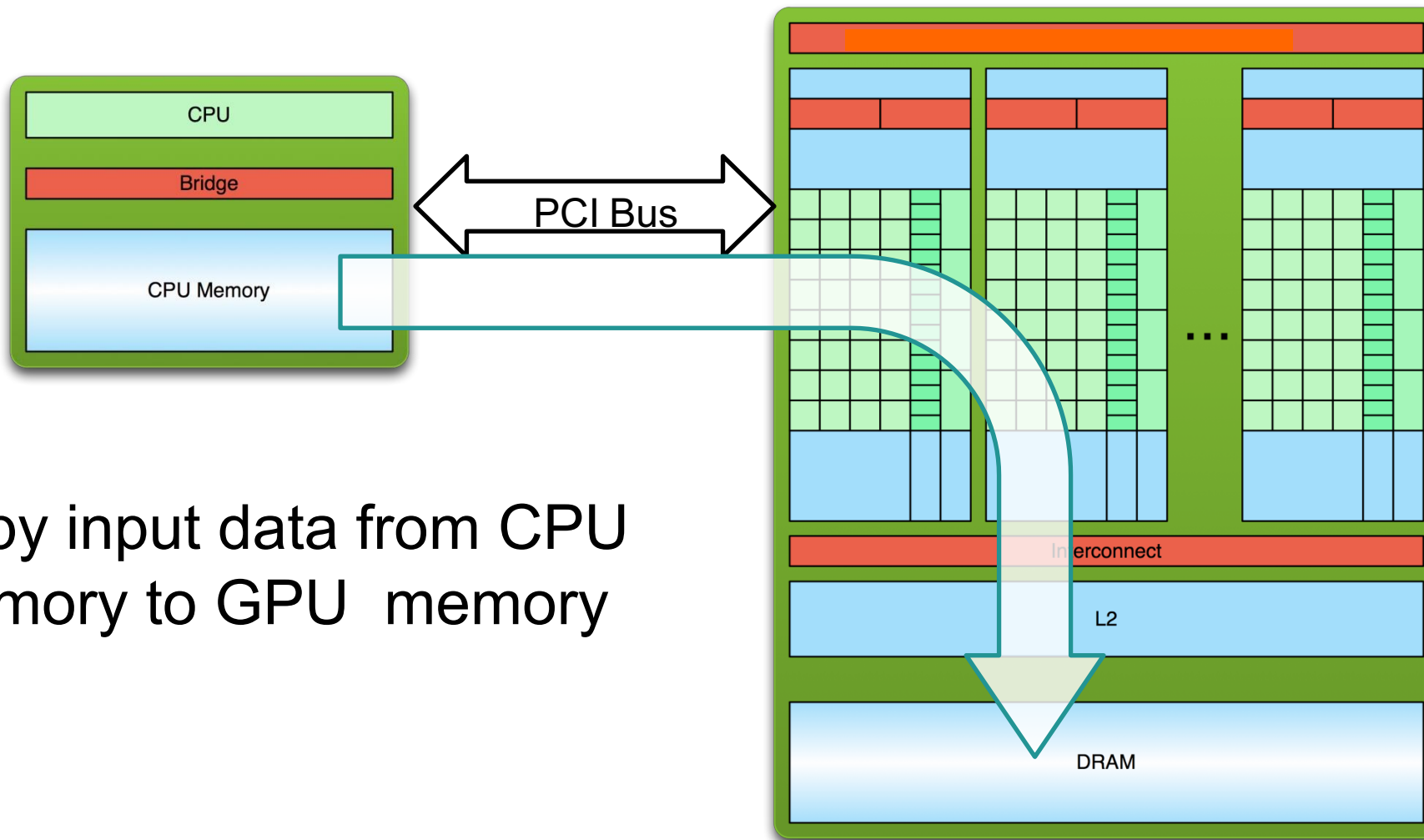
serial code

parallel code

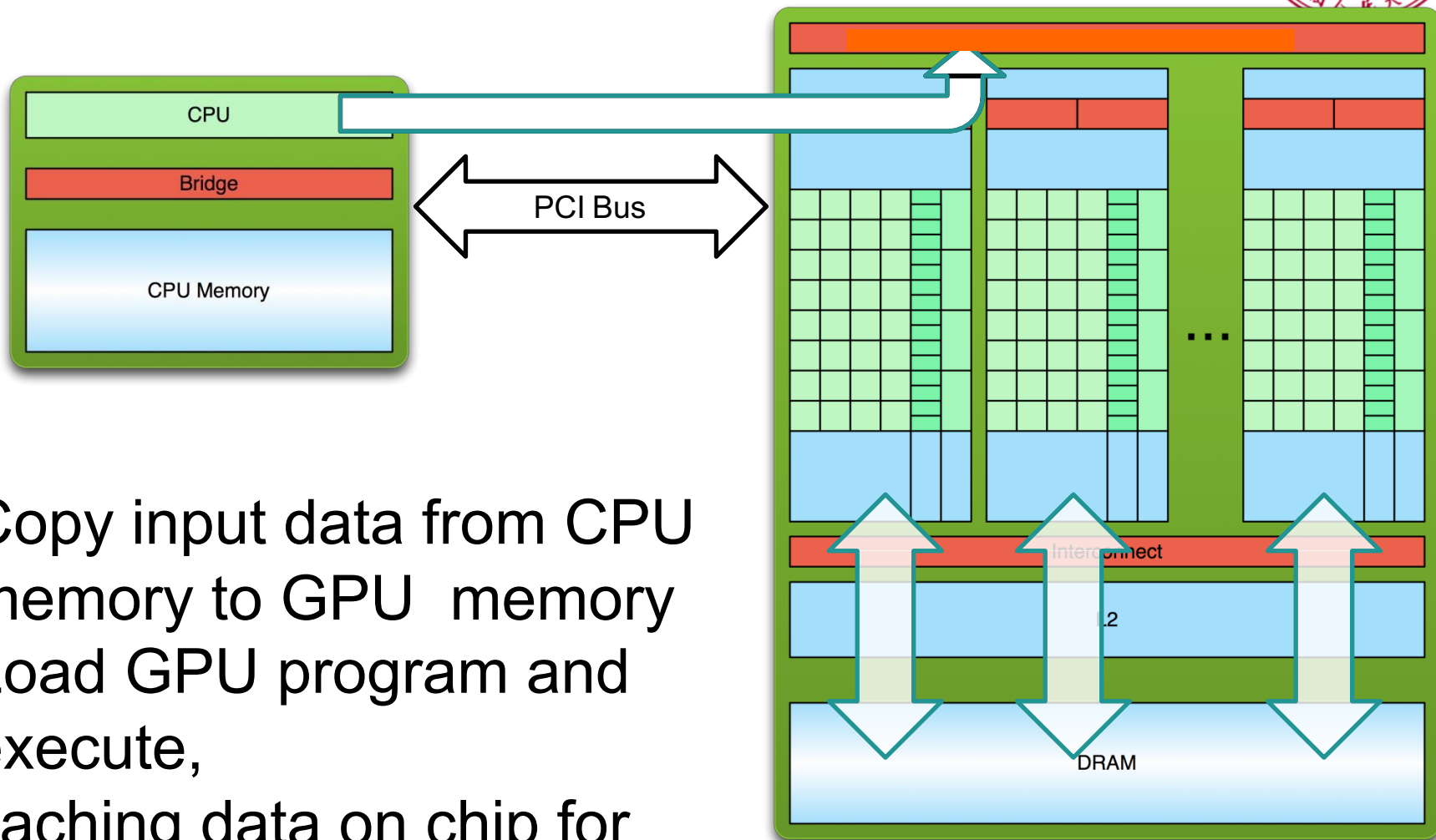
serial code



简单的处理流程

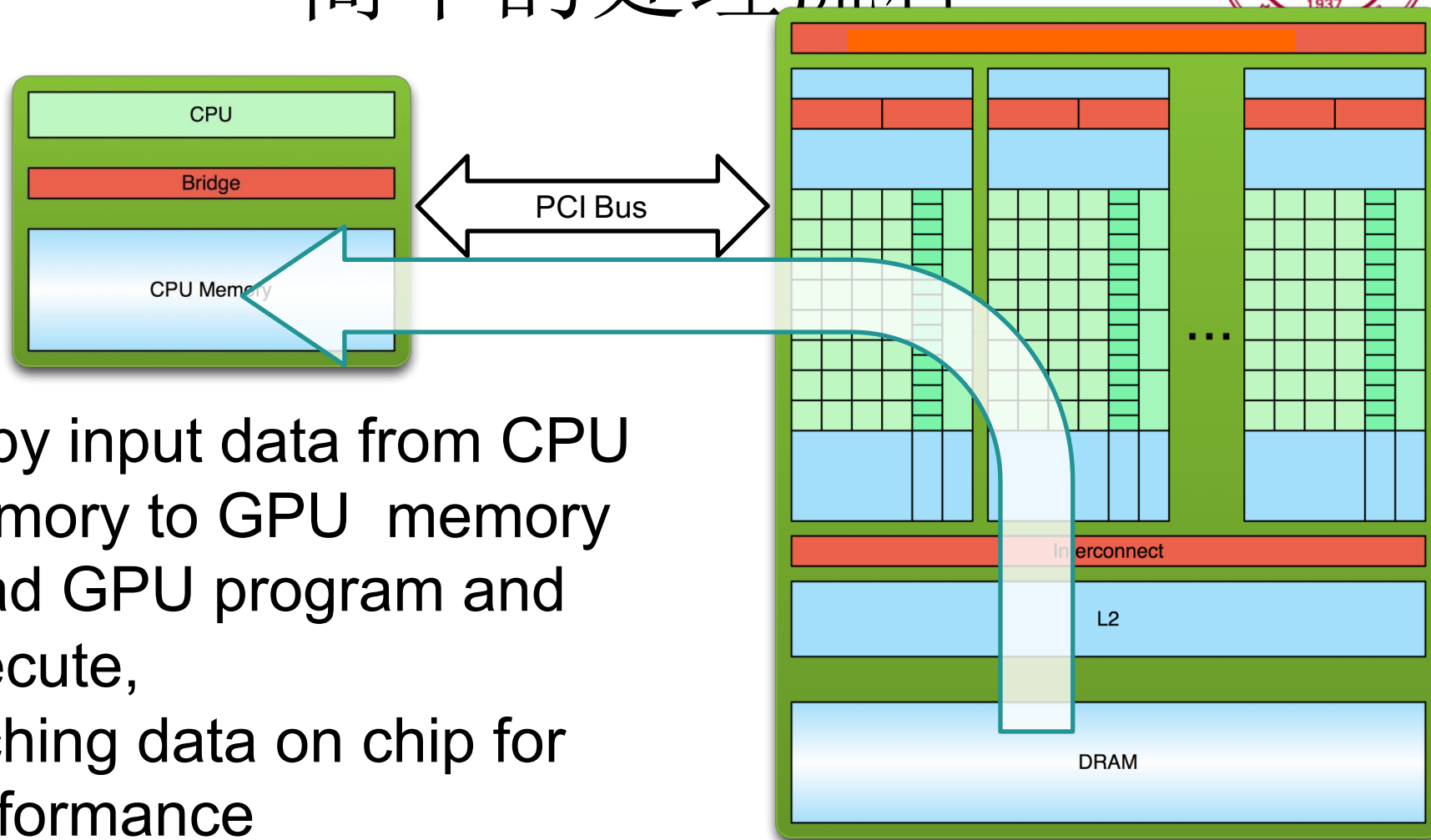


简单的处理流程



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

简单的处理流程



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance
3. Copy results from GPU memory to CPU memory



Hello Word!

```
int main(void) {  
    printf("Hello World!\n");  
    return 0;  
}
```

- 标准运行在host的C程序
- NVIDIA编译器能够编译不使用GPU的程序

Output:

```
$ nvcc hello_world.cu  
$ a.out Hello World!  
$
```



Hello World! 使用device

```
__global__ void mykernel(void) {  
}
```

```
int main(void) {  
    mykernel<<<1,1>>>();  
    printf("Hello World!\n");    return 0;  
}
```

- 增加了少量的代码



Hello World! 使用device

```
__global__ void mykernel(void) {  
}
```

- CUDA C / C++关键字__global__表示函数:
- 在device上运行
- Host代码调用
- nvcc编译器会将host和device代码进行区分
- Device函数（mykernel()）由NVIDIA编译器编译
- Host函数（main()）由标准host端编译器编译
 - gcc等



Hello World! 使用device

```
mykernel<<<1,1>>>();
```

- <<< >>>表示host调用device端的代码
 - 也是“kernel launch”
 - 稍后介绍（1，1）
- 开启GPU计算函数就是这么简单！



Hello World! 使用device

```
__global__ void mykernel(void) {  
}
```

```
int main(void) {  
    mykernel<<<1,1>>>();  
    printf("Hello World!\n");    return 0;  
}
```

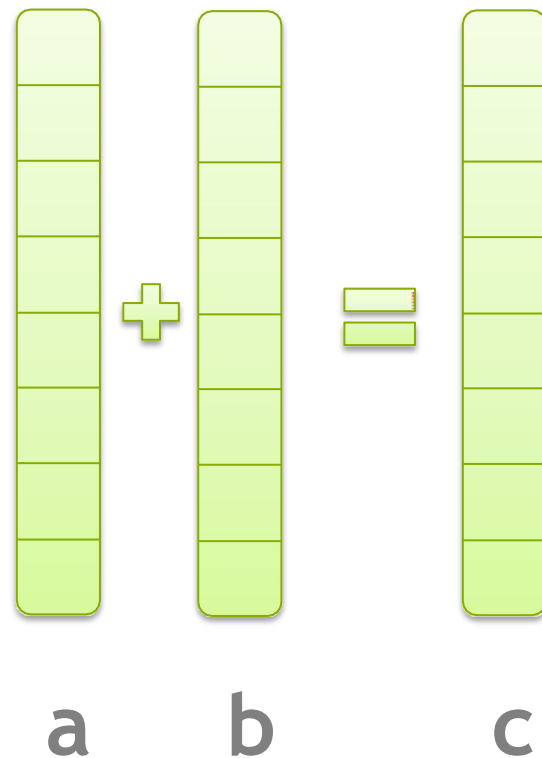
- mykernel()什么也没做

Output:

```
$ nvcc hello.cu  
$ a.out  
Hello World!  
$
```

CUDA并行编程

- CUDA计算应该调用大量的线程！
- 以向量加法为例进行探索





Device端概念

```
__global__ void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}
```

- 两个整数相加的kernel
- __global__是一个CUDA C / C++关键字
 - add()会运行在device端
 - add()由host调用



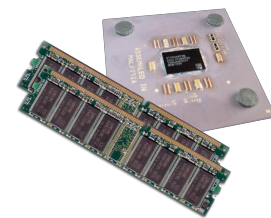
Device端概念

```
__global__ void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}
```

- 我们使用指针表示变量
- add()运行在device端，a、b、c必须指向设备内存
- 需要在GPU上分配内存

内存管理

- host和device内存是彼此独立的单元
 - Device指针指向GPU内存
 - 可能从host传入
 - 可能从host取消引用
 - Host指针指向CPU内存
 - 可能从host传入
 - 可能从host取消引用
- 涉及device内存的CUDA API
 - `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`
 - 和 C 语言中的 `malloc()`, `free()`, `memcpy()` 类似





Device端部分

```
__global__ void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}
```

add() kernel回顾

- 接下来介绍如何写main() ...



```
int main(void) {
```

```
    // host copies of a, b, c
```

```
    int a, b, c;
```

```
    // device copies of a, b, c
```

```
    int *d_a, *d_b, *d_c;
```

```
    int size = sizeof(int);
```

```
    // Allocate space for device copies of a, b, c
```

```
    cudaMalloc((void **) &d_a, size);
```

```
    cudaMalloc((void **) &d_b, size);
```

```
    cudaMalloc((void **) &d_c, size);
```

```
    // Setup input values
```

```
    a = 2;
```

```
    b = 7;
```



// Copy inputs to device

```
cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);  
cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);
```

// Launch add() kernel on GPU

```
add<<<1,1>>>(d_a, d_b, d_c);
```

// Copy result back to host

```
cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);
```

// Cleanup

```
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);  
return 0;
```

```
}
```



回顾

- CUDA C / C++关键字 `__global__`
- `<<< >>>` 表示host调用device端的代码
- `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`



CONCEPTS

RUNNING IN PARALLEL

Heterogeneous
Computing

Blocks

Threads

Indexing

Shared memory

`_syncthreads()`

Asynchronous
operation

Handling errors

Managing devices



开启多线程

- GPU计算应开启大量的线程进行并行
- 如何在GPU上进行并行运算呢？

`add<<<1, 1>>>();`

`add<<<N, 1>>>();`

- 将add()并行执行了N次



GPU上的Vector Add

- 通过将add()并行运行进行向量加法
- 术语：block，add()的一次并行执行
- 一组block构成了grid
- 每个block可以通过blockIdx.x获得ID，每个block处理一个数

```
__global__ void add(int *a, int *b, int *c){  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```



GPU上的Vector Add

```
__global__ void add(int *a, int *b, int *c){  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

- 在device端，每个block并行执行如下代码

Block 0

`c[0] = a[0] + b[0];`

Block 1

`c[1] = a[1] + b[1];`

Block 2

`c[2] = a[2] + b[2];`

Block 3

`c[3] = a[3] + b[3];`

...



GPU Vector Add: main()

```
#define N 512

int main(void) {
    int *a, *b, *c;           // host copies of a, b, c
    int *d_a, *d_b, *d_c;     // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```



GPU Vector Add: main()

```
// Copy inputs to device
cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU with N blocks
add<<<N,1>>>(d_a, d_b, d_c);

// Copy result back to host
cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

// Cleanup
free(a); free(b); free(c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}
```



回顾（1/2）

- host和device的区别
 - Host – CPU
 - Device – GPU
- 使用__global__来表示一个函数运行在device上
 - 在device上执行
 - 从host调用
- 从host端代码传参数到device函数中



回顾（2/2）



- 基本的设备端内存管理
 - `cudaMalloc()`
 - `cudaMemcpy()`
 - `cudaFree()`
- 开启（launch）并行计算函数(kernel)
 - 使用 “`add<<<N, 1>>> (...);`” 开启N份`add()`
 - 使用`blockIdx.x`访问相应的block



线程

THREADS

Heterogeneous Computing

Blocks

Threads

Indexing

Shared memory

`_syncthreads()`

Asynchronous operation

Handling errors

Managing devices



CUDA 线程

- 一个block可以分为许多个并行线程
- 下面将把add()由block parallel修改为thread parallel

Using blocks:

```
__global__ void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

```
add<<<N,1>>>>(d_a, d_b, d_c);
```

Using threads:

```
__global__ void add(int *a, int *b, int *c) {  
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];  
}
```

```
add<<<1,N>>>>(d_a, d_b, d_c);
```



同时考虑 threads和blocks

Heterogeneous Computing

Blocks

Threads

Indexing

Shared memory

`_syncthreads()`

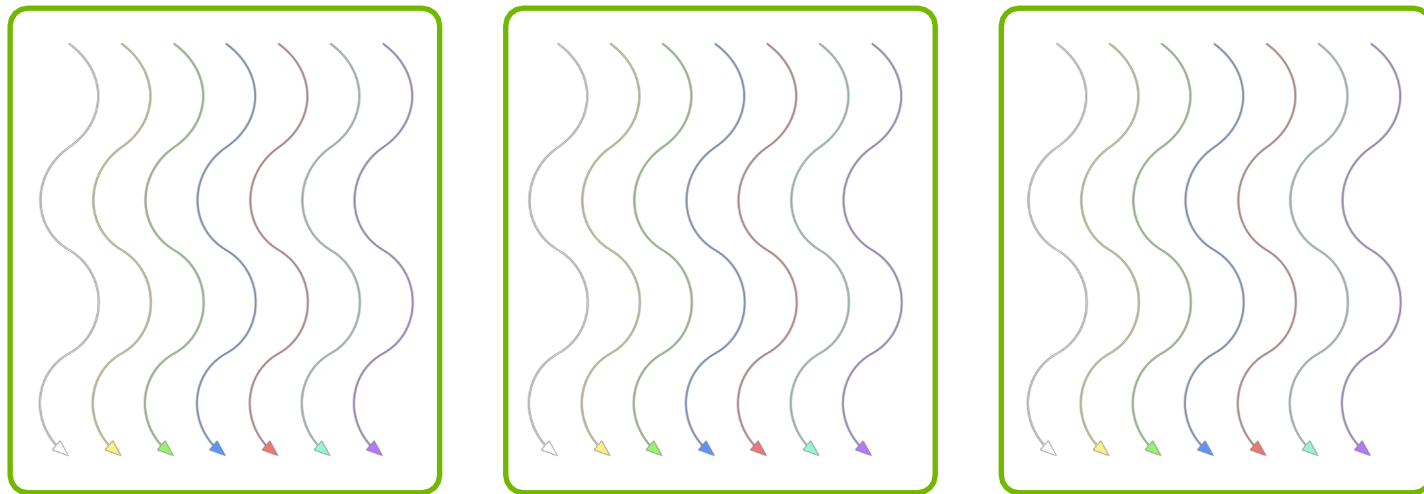
Asynchronous operation

Handling errors

Managing devices

同时使用block和thread

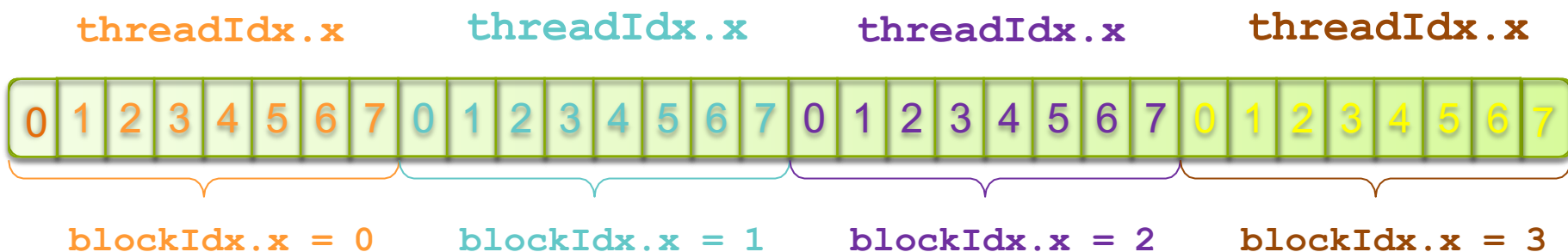
- 并行向量加法的方法：
 - 多个block，每个block一个thread
 - 一个block，包含多个threads
- 如果同时使用blocks和threads





同时使用block和thread 线程编号（index）

- 和之前使用blockIdx.x和threadIdx.x用法一样
 - 考虑每个线程处理矩阵中的一个元素（8 threads/block）



- 假设每个block有M个线程，那么每个线程的编号为：

```
int index = threadIdx.x + blockIdx.x * M;
```



同时使用block和thread 线程编号 (index)

- 可以使用内建的变量blockDim.x获得每个block的线程数目

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```

- 同时使用block和thread的add()版本

```
__global__ void add(int *a, int *b, int *c) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    c[index] = a[index] + b[index];  
}
```

- main()函数中需要哪些修改？



同时使用block和thread main()

```
#define N (2048*2048)
#define THREADS_PER_BLOCK 512
int main(void) {
    int *a, *b, *c;           // host copies of a, b, c
    int *d_a, *d_b, *d_c;     // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```



同时使用block和thread main()

```
// Copy inputs to device
cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU
add<<<N/THREADS_PER_BLOCK,THREADS_PER_BLOCK>>>(d_a, d_b, d_c);

// Copy result back to host
cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

// Cleanup
free(a); free(b); free(c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}
```



对于任意大小的向量

- 通常问题的规模不会是blockDim.x的整数倍
- 避免数据访问越界

```
__global__ void add(int *a, int *b, int *c, int n) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    if (index < n)  
        c[index] = a[index] + b[index];  
}
```

- kernel launch也要相应调整

```
add<<<(N + M-1) / M, M >>>(d_a, d_b, d_c, N);
```



为什么要有threads的概念？

- threads似乎用处不大
 - 增加了一层复杂性
- 和block不同，threads有如下机制：
 - communicate 通信
 - synchronize 同步

新的例子来解释



线程间的交互

Heterogeneous Computing

Blocks

Threads

Indexing

Shared memory

`_syncthreads()`

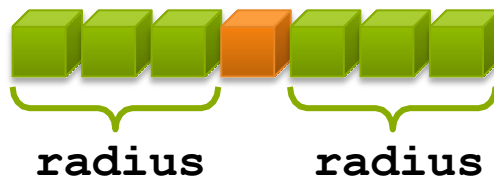
Asynchronous operation

Handling errors

Managing devices

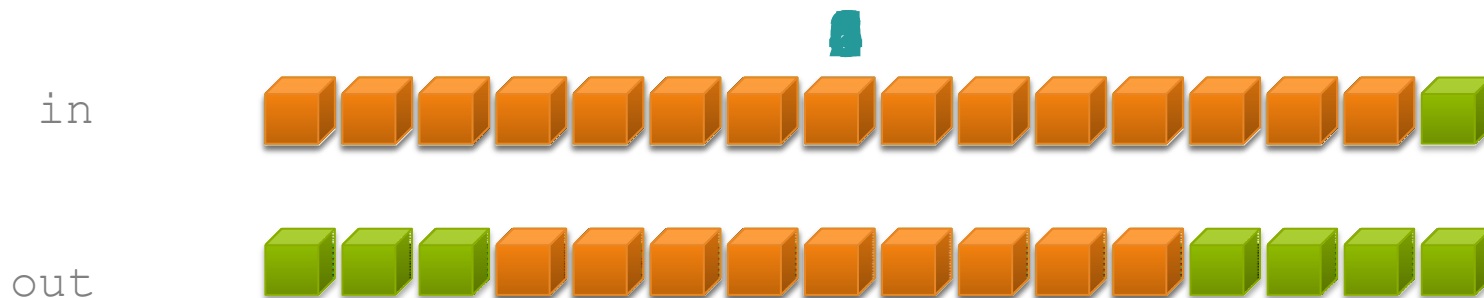
1D Stencil

- **Stencil:** 每个位置的数值都是它周围一定范围内所有数的加合
- Each output element is the sum of input elements within a radius
- 考虑将**1D stencil**应用到**1D**矩阵
- 如果半径是**3**，那么每个输出元素的值是**7**个输入元素的加合



block内部的实现

- 每个线程负责计算一个元素
 - 每个block有blockDim.x个元素
- 每个输入元素会被读很多次（read）
 - 如果半径是3，那么每个输入元素会被读7次



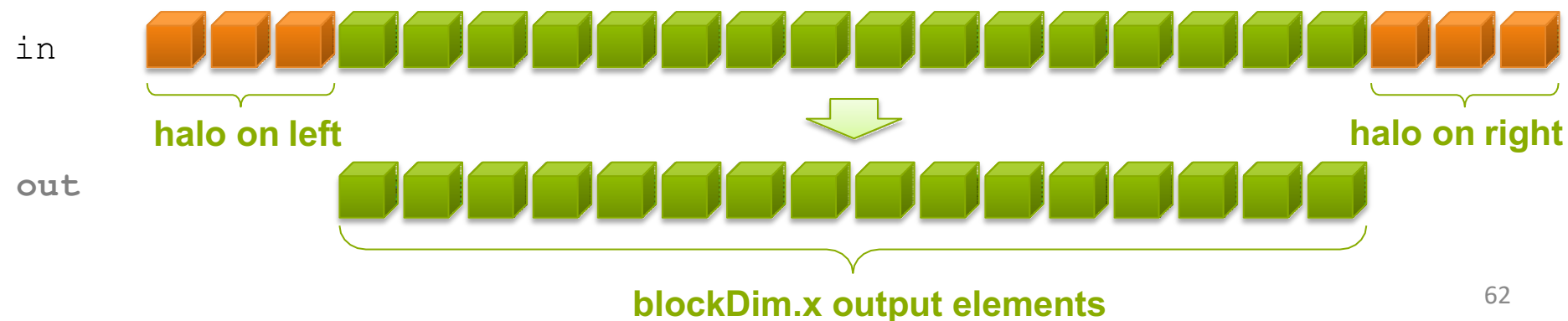


线程间共享数据

- 同一个block内部，线程间通过shared memory共享数据
- shared memory是片上用户定义的快速存储
- 使用__shared__关键字为每个block分配空间
- 不同block的线程不可见

stencil使用shared memory

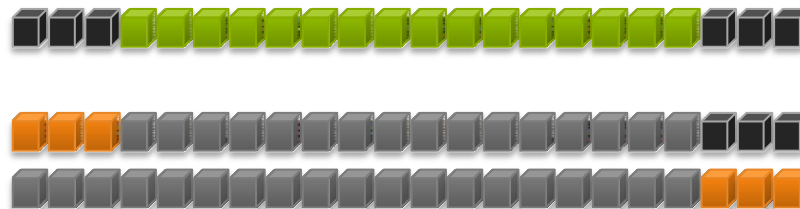
- 将数据还存在shared memory中
 - 从global memory中读入 ($\text{blockDim.x} + 2 * \text{radius}$) 个元素存入shared memory中
 - 计算 blockDim.x 个输出结果元素
 - 将 blockDim.x 个结果元素写回到global memory
- 每个block在边界需要多读radius个元素





Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {  
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];  
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;  
    int lindex = threadIdx.x + RADIUS;  
  
    // Read input elements into shared memory  
    temp[lindex] = in[gindex];  
    if (threadIdx.x < RADIUS) {  
        temp[lindex - RADIUS] = in[gindex - RADIUS];  
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];  
    }  
}
```





Stencil Kernel(接上页)

```
// Apply the stencil
```

```
int result = 0;
```

```
for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
```

```
    result += temp[lindex + offset];
```

```
// Store the result
```

```
out[gindex] = result;
```

```
}
```



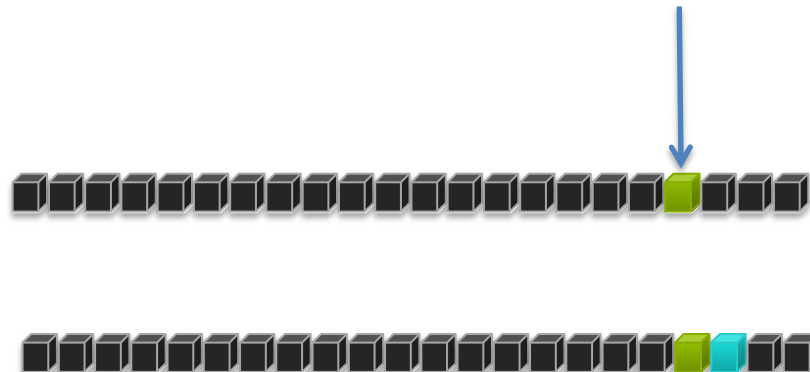
Stencil Kernel(接上页)

```
// Apply the stencil  
int result = 0;  
for (int offset = -RADIUS ; offset <= RADIUS ; offset++)  
    result += temp[lindex + offset];  
  
// Store the result  
out[gindex] = result;  
}
```

有没有问题？

DATA RACE!

```
temp[lindex] = in[gindex];  
if (threadIdx.x < RADIUS) {  
    temp[lindex - RADIUS] = in[gindex - RADIUS];  
    temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];  
}  
  
int result = 0;  
for (int offset = -RADIUS ; offset <= RADIUS ; offset++)  
    result += temp[lindex + offset];
```





syncthreads()

- `void syncthreads();`
- 同步同一个block内的所有线程
 - 用来防止RAW/WAR/WAW竞争
- 所有的线程必须在barrier等待
 - 针对block内所有线程



Stencil Kernel

```
global void stencil_1d(int *in, int *out) {  
    shared  int temp[BLOCK_SIZE + 2 * RADIUS];  
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;  
    int lindex = threadIdx.x + radius;  
  
    // Read input elements into shared memory  
    temp[lindex] = in[gindex];  
    if (threadIdx.x < RADIUS) {  
        temp[lindex - RADIUS] = in[gindex - RADIUS];  
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];  
    }  
  
    // Synchronize (ensure all the data is available)  
    syncthreads();  
}
```



Stencil Kernel

```
// Apply the stencil  
int result = 0;  
for (int offset = -RADIUS ; offset <= RADIUS ; offset++)  
    result += temp[lindex + offset];  
  
// Store the result  
out[gindex] = result;  
}
```



回顾 (1/2)

- 开启多个并行线程
- `kernel<<<N,M>>>(...)`; 开启N个block，每个block有M个线程
- 使用`blockIdx.x`访问grid中block的编号
- 使用`threadIdx.x`访问block中线程的编号
- 向各线程分配数据

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```



回顾（2/2）

- 使用__shared__声明放在shared memory中的变量或数组
 - 数据由同一个block内的线程共享
 - 对其他block内的线程不可见
- 使用__syncthreads()做barrier操作
 - 防止数据竞争



设备管理

Heterogeneous Computing

Blocks

Threads

Indexing

Shared memory

`_syncthreads()`

Asynchronous operation

Handling errors

Managing devices



host和device间的交互

- kernel的开启是异步的（**asynchronous**）
 - 开启后迅速转换到CPU端
- CPU需要在处理结果前进行同步
- cudaMemcpy()
 - 直到copy完成CPU才能继续后面的操作
 - 在所有的CUDA调用完成后才能执行copy操作
- cudaMemcpyAsync()
 - 异步操作，不阻塞CPU
- cudaDeviceSynchronize()
 - 阻塞CPU直到所有的CUDA调用结束才继续执行



获得错误信息

- 所有的**CUDA API**调用都会返回错误编号 (`cudaError_t`)
- **API**自身的错误
- 或者之前的一个异步操作（如**kernel**）
- 获得最后一个错误
- `cudaError_t cudaGetLastError(void)`
- 语言描述

```
char *cudaGetErrorString(cudaError_t)
printf("%s\n",
cudaGetErrorString(cudaGetLastError(
))) ;
```



设备管理

- 应用程序查找、使用GPU
 - `cudaGetDeviceCount(int *count)`
 - `cudaSetDevice(int device) cudaGetDevice(int *device)`
 - `cudaGetDeviceProperties(cudaDeviceProp *prop, int device)`
- 多个CPU进程可以共享一个GPU
- 单个CPU进程可以控制多个GPU
 - `cudaSetDevice(i)`选择设备



总结

- 如何开启CUDA C/C++ kernel
 - `__global__` , `blockIdx.x`, `threadIdx.x`, `<<<>>>`
- 管理GPU内存
 - `cudaMalloc()`, `cudaMemcpy()`, `cudaFree()`
- 控制通信（communication）和同步（synchronization）
 - `__shared__` , `__syncthreads()`
 - `cudaMemcpy()` vs `cudaMemcpyAsync()`, `cudaDeviceSynchronize()`

谢谢！



中國人民大學
RENMIN UNIVERSITY OF CHINA